



§ 17. 继承与派生 – 画内存图并分析

要求：分析下面的程序为什么在VS下运行会出错

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    ~A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b);    //复制构造
    A a2;
    a2 = b;     //赋值
}
```

- ★ main函数中红色的三句，每句均要画出当前的内存图（为方便阅读，一句可以多页），分析为什么会出错
- ★ 可参考第13模块 P. 90-92、P. 113-115，第16模块 P. 8-P. 14
- ★ 不允许手写、手写后贴图，转换为pdf后在“文档作业”中提交（2024. 12. 19前）



B类定义对象时默认调用基类A的构造函数，给char指针s分配20字节的空间

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b);    //复制构造
    A a2;
    a2 = b;     //赋值
}
```

2000	3000
b	0
2011	0

3000	未知
3019	

2000	3000
b	15
2011	0



3000	未知
3019	

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    ~A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b);    //复制构造
    A a2;
    a2 = b;     //赋值
}
```

2000	3000
b	15
2011	0

2000	3000
b	15
2011	0

8字节整体拷贝

2100	3000
a1	
2107	15

赋值兼容规则生效，将派生类中基类对应成员拷贝给新的基类，其余部分丢弃。同时，由于没有重载复制构造，系统调用默认复制构造函数（浅拷贝），只将指针的值拷贝过去，没有申请新的空间，两个指针指向同一个空间。



3000	未知
3019	

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    ~A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b);    //复制构造
    A a2;
    a2 = b;     //赋值
}
```

2000	3000
b	15
2011	0

2100	3000
a1	15
2107	

2000	3000
b	15
2011	0

8字节整体拷贝

2200	3000
a2	15
2207	

赋值兼容规则生效，将派生类中基类对应成员拷贝给新的基类，其余部分丢弃。
同时，由于没有重载赋值运算符，系统调用默认赋值运算（浅拷贝），只将指针的值拷贝过去，没有申请新的空间，两个指针指向同一个空间。



程序结束，开始销毁变量/对象

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    ~A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b);    //复制构造
    A a2;
    a2 = b;     //赋值
}
```

2000	?
b	15
2011	0

2100	3000
a1	15
2107	

2200	3000
a2	15
2207	

首先销毁b,派生类对象在析构时会默认调用基类的析构函数。A类的析构函数会释放字符指针指向的空间。在本例中，内存3000-3019的空间已经被释放。

3000	未知
3019	



程序结束，开始销毁变量/对象

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    ~A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b); //复制构造
    A a2;
    a2 = b; //赋值
}
```

2000	?
b	15
2011	0

3000	未知
3019	

2100	3000
a1	
2107	15

2200	3000
a2	
2207	15

然后销毁a1。

a1对象在销毁时，调用A类的析构函数，释放字符指针s指向的空间3000-3019。

但是，在b销毁时，该空间已经被释放。再次释放同一块内存，程序错！



程序结束，开始销毁变量/对象

```
#include <iostream>
using namespace std;
class A {
private:
    char* s;
public:
    int a;
    A() { s = new char[20]; }
    ~A() {
        cout << "A析构" << endl;
        delete s;
    }
};
class B :public A {
public:
    int b;
    ~B() { cout << "B析构" << endl; }
};
int main()
{
    B b;
    b.a = 15;
    A a1(b); //复制构造
    A a2;
    a2 = b; //赋值
}
```

2000	?
b	15
2011	0

2100	3000
a1	15
2107	

2200	3000
a2	15
2207	

3000	未知
3019	

最后销毁a2。

a2对象在销毁时，调用A类的析构函数，释放字符指针s指向的空间3000-3019。

但是，在b销毁时，该空间已经被释放。再次释放同一块内存，程序错！

（实际运行过程中，程序在执行到销毁a1，两次释放同一块空间时就会报错终止，不会执行到销毁a2这一步）